

PORTIX

Guía completa del proyecto

Semáforo de salida para transportistas del puerto de Veracruz: qué camión sale, cuál espera y qué cita hay que mover, antes de que el camión llegue a hacer fila.

Equipo Los Ubuntus 2.0 · Olimpia, Renata, Othon, Brayan y Charly
Hackathon Veracruz Tech Week 2026 · Reto 1: «Puerto que fluye»
12 de noviembre de 2026

Demo en vivo: https://losubuntus2-0.onrender.com/

Contenido

1. PORTIX en una página
2. La problemática que atacamos
3. Persona usuaria y beneficiarios
4. Cómo funciona: toda la funcionalidad
5. Las reglas del semáforo, paso a paso
6. Uso de la inteligencia artificial
7. El giro del reto: aviso por SMS o voz
8. Tecnologías que usamos y por qué
9. Arquitectura y archivos del código
10. Qué es real y qué es simulado
11. Alcance: qué hace y qué no
12. Empresas y organizaciones que podrían ayudar
13. Qué se necesita para correrlo, desplegarlo y llevarlo a producción
14. Guion de la demo
15. Preguntas probables del jurado
16. Fuentes

Para qué sirve esta guía. Una de las reglas del hackathon es que **cualquier integrante debe poder explicar cualquier parte** del proyecto. Esta guía junta en un solo lugar qué problema resolvemos, cómo funciona cada pantalla, por qué elegimos cada tecnología, qué es real y qué es simulado, y qué preguntas nos pueden hacer. Léanla completa antes de presentar.

1. PORTIX en una página

Antes de que un camión salga hacia el puerto de Veracruz, PORTIX le asigna un **color** a su contenedor:

Color	Significa	Qué hace PORTIX
Verde · Sale	Documentos completos y el camión llega dentro de la ventana de su cita.	Le dice al despachador a qué hora salir, y le avisa al operador por SMS.
Amarillo · Espera	Falta un documento, o la hora de salida no cuadra, pero todavía hay tiempo.	Dice qué falta y la hora límite para salir. Avisa a la agencia aduanal por el chat.
Rojo · Reprogramar	Ni saliendo ahora llega a tiempo a la ventana de su cita.	Sugiere mover o cancelar la cita antes de que cierre la ventana, para evitar el cargo de No Show.

La idea central: el problema del puerto no es solo de capacidad, es de **coordinación e información**. Muchos camiones salen sin los papeles listos o a una hora en la que no van a alcanzar su cita, hacen horas de fila y al final los regresan o pagan una multa. PORTIX detecta eso **antes de que el camión salga**.

La métrica que nos importa es la espera: horas de fila evitadas y citas no perdidas. La revisión de documentos con IA y el chat con la agencia son medios para llegar a eso, no el producto en sí.

Lo construimos en fases, y cada fase termina en una demo que funciona:

Fase	Qué agregó
F1 · Tablero base	Contenedores del día (sintéticos), checklist de documentos y la regla de color.
F2 · Estimación de tiempo	Congestión por hora (simulada), llegada estimada contra la ventana, probar otra hora de salida, reprogramar y contador de No Show evitados.
F3 · Revisión con IA	Subir un PDF o una foto; Gemini extrae los datos y los compara con los del viaje.
F4 · Avisos a la agencia	Chat simulado; la IA interpreta lo que responde la agencia y actualiza el checklist.
F5 · Producto y verificación	Tres vistas, agregar contenedores (y llenarlos con IA desde la carta porte), KPIs de impacto, regla del rojo corregida, reinicio de la demo y 46 pruebas automáticas.
Giro · SMS o voz	Aviso al operador sin smartphone por SMS (real con Twilio) o mensaje de voz.

2. La problemática que atacamos

Cada día entran al puerto de Veracruz cerca de **2,000 tractocamiones**. En septiembre de 2026, un camión pasó en promedio **12 h 38 min** dentro del puerto: el peor tiempo de los cuatro grandes puertos del país, y el doble que en junio (6 h 15 min).

Dato	Valor	Fuente
Tiempo promedio en el puerto	Jun. 6:15 · Jul. 7:13 · Ago. 11:31 · Sep. 12:38	ITPAP, T21
Desglose de septiembre 2026	Aduana 6:09 · Espera 3:49 · Maniobras 2:39	ITPAP, T21 (30 oct. 2026)
Flujo diario	~2,000 tractocamiones	T21 / CANACAR
Costo modelado de las demoras	~8.9 millones de pesos al día (agosto); es un costo modelado, no contable	T21
Tolerancia de la cita	2 h antes y 1 h después del horario	ICAVE, aviso ICA-2022-013
Cargo por No Show	$999 MXN por contenedor (desde el 2 de junio de 2025)	ICAVE, aviso ICA-2025-009
Casos extremos	17 a 24 h en la ruta fiscal	CANACAR vía Plumas Libres (jun. 2026)

Tres problemas encadenados

1. El camión sale sin los documentos listos. La aduana se lleva 6 h 09 min de las 12 h 38 min. Terminales y transportistas coinciden en que buena parte de las demoras viene de documentos incompletos (pedimento sin pagar, carta porte con placas que no coinciden, folio de cita faltante), no solo de la capacidad del puerto.

2. La espera es impredecible. El promedio se duplicó en tres meses y hay casos de 17 a 24 horas. Con esa variación, el despachador no puede planear su día y manda camiones "a ver si pasan".

3. El camión pierde su cita y paga por ello. La cita en la terminal tiene una tolerancia de 2 h antes y 1 h después. Si el camión no llega y nadie canceló a tiempo, la terminal cobra $999 por contenedor (No Show).

> **Por qué un semáforo antes de salir:** los tres problemas se pueden detectar **en el patio de la transportista**, antes de que el camión arranque. Ahí todavía se puede corregir un documento, ajustar la hora de salida o cancelar la cita sin cargo. Una vez en la fila, ya es tarde.

Cuidado con las cifras

- No usamos "2,400 operadores al día" ni "CALT con capacidad para 650 camiones": son de 2023.
- No usamos "48 h de espera": la fuente no tiene fecha verificable.
- El ITPAP es un **promedio mensual**. No hay datos públicos por hora del día; por eso nuestra congestión por hora es **simulada** y así lo decimos.

3. Persona usuaria y beneficiarios

Usuario principal: el despachador de una transportista mediana que mueve contenedores desde el puerto. Coordina decenas de camiones al día por teléfono, WhatsApp y hojas de cálculo. Depende de la agencia aduanal para los documentos y de la terminal para las citas.

Por qué la transportista y no la agencia aduanal: la transportista es quien hace la fila, paga el diésel, las horas del operador y el cargo de No Show. Es quien más gana si el camión no sale cuando no debe.

Actor	Cómo participa	Qué gana
Despachador (usuario)	Usa el tablero: ve colores, reprograma citas, revisa documentos, agrega contenedores.	Menos camiones parados, menos multas, un plan del día confiable.
Agencia aduanal	Recibe los avisos de lo que falta y responde por el chat.	Menos llamadas; sabe qué es urgente.
Operador del camión	Recibe el color de su contenedor por SMS o llamada (giro).	Menos horas de fila; sabe si salir o no.
Vecinos de las zonas de acceso	Indirecto.	Menos camiones estacionados en sus calles.

4. Cómo funciona: toda la funcionalidad

La app tiene un **encabezado** y **tres vistas** (pestañas). El contenedor elegido se comparte entre las tres, y la URL cambia (#panel, #documentos, #chat) para que el botón Atrás funcione.

4.1 Encabezado

- **Hora de la demo (fija, 06:15):** la "hora actual" no es la del reloj real, para que la demo siempre dé los mismos colores sin importar a qué hora presentemos. Se configura con PORTIX_HORA_ACTUAL.
- **Estado de la IA:** "Gemini conectada" o "modo respaldo".
- **Estado de los avisos SMS/voz:** reales, con plantilla de prueba, o simulados.

- **Reiniciar demo:** regresa todo al estado inicial (contenedores, contador, chats y avisos). Pide confirmación.

4.2 Vista Panel (el tablero del día)

- **Resumen con 5 tarjetas:** cuántos salen, esperan y hay que reprogramar; **No Show evitados** con los pesos ahorrados ($999 cada uno; la tarjeta hace un pulso cuando sube), y **horas de fila evitadas** (estimación: camiones detenidos por documentos × 3 h 49 min, la espera promedio del ITPAP de septiembre).
- **Gráfica de congestión por hora** (simulada y etiquetada así): baja, media o alta de 06:00 a 17:00. Marca la hora de salida del contenedor elegido.
- **Lista de contenedores** ordenada por urgencia (rojo, amarillo, verde), con cita, salida, llegada estimada, documentos listos y acción sugerida. Si un contenedor cambia de color, su fila parpadea.
- **+ Agregar contenedor:** formulario con id (formato ABCD 123456-7), tipo de carga, terminal, cita, salida, operador, placas y teléfono. Los documentos empiezan pendientes. Botón **"Llenar desde carta porte (IA)"**: sube el PDF y Gemini llena los campos para que el despachador los revise (nunca guarda solo).

Al elegir un contenedor se abre el detalle:

- Color, motivo y **acción sugerida** (por ejemplo, "Adelantar la salida a las 10:22 o antes").
- Atajos a **Revisar documento con IA** y **Avisar a la agencia**.
- **Aviso al operador** (giro): celular básico dibujado con el SMS, botones Enviar SMS, Llamar y Escuchar aquí, e historial de avisos.
- **Barra de llegada contra la ventana:** muestra la ventana de la cita y dónde cae la llegada estimada.
- **Reprogramar** (solo en rojo): mover la cita a otra hora o cancelarla. Si se hace antes de que cierre la ventana, suma un No Show evitado y le manda un SMS automático al operador.
- **Probar otra hora de salida:** recalcula el color "¿y si sale a las 09:15?" sin guardar nada.
- **Datos del viaje** y **checklist de 5 documentos**: pedimento, carta porte, folio de cita, operador y placas, requisito de la carga. Cada uno en Listo, Falta, Con error o No aplica; al cambiarlo, el color se recalcula.

4.3 Vista Documentos con IA

- Lista compacta de contenedores, primero los que tienen documentos pendientes.
- Se sube una **carta porte o el comprobante del pedimento** (PDF, JPG o PNG, máx. 10 MB).
- Gemini extrae contenedor, placas, operador y número de pedimento, y los **compara con los datos del viaje**. Muestra una tabla de hallazgos (coincide / no coincide).
- El renglón del checklist queda en **Listo** si todo coincide o en **Con error** si algo no coincide o no se pudo leer. El color del contenedor se recalcula.
- Solo revisa **consistencia** entre el documento y el viaje; no valida nada contra el SAT.

4.4 Vista Chat con la agencia aduanal

- Bandeja con los contenedores que tienen pendientes. Al abrir uno, PORTIX ya escribió el primer aviso: qué falta.
- Alguien del equipo escribe como si fuera la agencia (el chat es **simulado**; no hay WhatsApp real).
- Gemini **interpreta** el mensaje en español coloquial: qué documento menciona, en qué estado queda, si quedó algo pendiente y qué tan segura está.

- **Regla estricta:** si la agencia dice "ya quedó pagado, al rato te paso el comprobante", el pedimento sigue **pendiente** hasta que se sube el comprobante y la IA lo revisa.
- **Si la IA tiene confianza baja** o el mensaje es un saludo, no cambia nada y pide revisarlo a mano.
- Se muestra el **antes y después** del color del contenedor.

5. Las reglas del semáforo, paso a paso

Viven en **backend/semaforo.py**. Son reglas simples y explicables, no un modelo de caja negra.

Cálculos

Concepto	Cómo se calcula
Ventana de la cita	De 2 h antes a 1 h después de la cita (regla de ICAVE). Cita 09:00: ventana 07:00 a 10:00.
Traslado del patio al puerto	45 min × la congestión de la hora de salida: baja ×1.0 (45 min), media ×1.3 (59 min), alta ×1.5 (68 min). Se redondea hacia arriba: mejor estimar de más.
Salida efectiva	La salida planeada, o la hora actual si la planeada ya pasó (el camión sigue en el patio; lo más temprano que puede salir es ahora).
Llegada estimada	Salida efectiva + traslado.
Última salida posible	La hora más tarde a la que puede salir y aún llegar antes del cierre de la ventana. Se busca minuto a minuto, porque el traslado cambia según la hora.

Color: gana la primera regla que se cumple

#	Condición	Color y acción
1	La cita fue cancelada	Amarillo: solicitar una nueva cita.
2	La hora actual ya pasó la última salida posible (ni saliendo ahora llega)	**Rojo:** mover o cancelar la cita antes del cierre. Si la ventana ya cerró, se dice que el cargo ya no se puede evitar.
3	Algún documento en Falta o Con error	Amarillo: resolver y salir a más tardar a la última salida posible.
4	Con la salida planeada llegaría después del cierre	Amarillo: adelantar la salida (todavía alcanza).
5	Llegaría antes de que abra la ventana	Amarillo: salir a partir de cierta hora (si no, espera formado).
6	Todo lo demás	**Verde:** salir a la hora planeada.

Por qué el rojo depende de la hora actual. En una versión anterior, el rojo comparaba la salida *planeada* contra la ventana. A las 06:15 marcaba rojo una cita de las 13:00 que llegaba de sobra saliendo antes. Lo corregimos: rojo significa "ya no llega a tiempo", y si todavía alcanza es amarillo con la acción de adelantar la salida. Hay pruebas automáticas para cada regla y para los bordes exactos de la ventana.

No Show evitado: suma 1 (y $999) solo si el contenedor estaba en **rojo** y la cita se movió o canceló **antes de que cerrara su ventana**. Mover una cita verde no cuenta.

Datos de la demo a las 06:15: 9 contenedores sintéticos: 3 verdes, 4 amarillos (tres por documentos y uno por salir tarde) y 2 rojos reales (citas de 05:30 y 05:45 que ya no alcanzan, pero cuya ventana sigue abierta para cancelar sin cargo).

6. Uso de la inteligencia artificial

Usamos IA en dos sentidos: **dentro del producto** (Gemini) y **para construirlo** (Claude Code).

6.1 IA dentro de PORTIX (Gemini Flash)

Función (backend/ia.py)	Qué hace	Dónde se ve
revisar_documento()	Lee un PDF o una foto de la carta porte o del pedimento y compara sus datos con los del viaje.	Vista Documentos
interpretar_mensaje()	Entiende lo que escribe la agencia en español coloquial y decide qué documento cambia y a qué estado.	Vista Chat
extraer_datos_viaje()	Lee una carta porte y llena el formulario de un contenedor nuevo.	Agregar contenedor

Decisiones técnicas que conviene saber explicar:

- **Salida con esquema JSON:** a Gemini le pasamos la forma exacta de la respuesta (modelos Pydantic). Así nunca contesta texto libre que haya que adivinar, y lo validamos antes de usarlo.
- **Modelos alternos:** GEMINI_MODEL puede traer varios modelos separados por coma. Si el primero está saturado, se intenta el siguiente.
- **Límite de 15 segundos por intento** y razonamiento bajo: son tareas de extraer y comparar, no de pensar mucho. En una demo en vivo es mejor un respaldo que una pantalla congelada.
- **Respuestas de respaldo declaradas:** si no hay key, se acaba el límite gratuito o falla la red, ia.py no truena: regresa una respuesta de respaldo con la misma forma, y la pantalla muestra la etiqueta "Respuesta de respaldo". (El respaldo de documentos decide por el nombre del archivo; el del chat usa palabras clave.)
- **Seguridad en el chat:** con confianza baja la IA no cambia nada; la regla estricta evita marcar como listo un documento que todavía no llega.
- **Un solo archivo:** todo lo de Gemini vive en ia.py. Para cambiar de proveedor (por ejemplo a Sintergica o Groq) solo se toca ese archivo.

6.2 IA para construir el proyecto

Usamos **Claude Code** para investigar y validar fuentes, comparar usuarios y soluciones, escribir el código por fases, las pruebas, el despliegue y una revisión del proyecto como si fuera evaluador. Cada uso relevante está en la **bitácora de IA** del README (fecha, herramienta, para qué y decisión clave), como lo pide el hackathon. Todo el código fue revisado por el equipo y está comentado para que cualquiera lo pueda explicar.

7. El giro del reto: aviso por SMS o voz

> **El giro:** "Un transportista no tiene smartphone moderno: la solución debe avisarle por SMS o por mensaje de voz. Debe aparecer en la demo final."

Cómo lo atendimos: el semáforo ya no se queda en la pantalla del despachador; llega hasta la cabina del camión, a cualquier celular básico. El código vive en **backend/avisos.py**, aislado igual que Gemini en ia.py.

- **SMS:** plantilla fija según el color, por ejemplo: "PORTIX TGHU 174826-8: ROJO, NO SALGAS. Ya no llegas a la cita de las 05:45...". Siempre cabe en **160 caracteres y va sin acentos**: una sola letra como "á" cambia la codificación del SMS (de GSM-7 a UCS-2) y el límite baja a 70 caracteres, así que el mensaje se partiría. Usamos plantilla y no IA para que el aviso nunca falle.
- **Voz:** el mismo aviso en frases completas; **deletrea el número del contenedor** (para compararlo con la puerta) y lo **repite**. Útil si el operador va manejando o lee con dificultad.
- **Automático:** al mover o cancelar una cita, el operador recibe el SMS sin que nadie lo pida. También se puede mandar a mano desde el detalle.
- **Privacidad:** los teléfonos de los datos son ficticios (229 000 01xx). Los avisos reales solo van al TELEFONO_DEMO (el celular de un integrante que aceptó usarlo), que vive en variables de entorno y en pantalla se muestra enmascarado (***1234).

Lo que verificamos con Twilio

Conectamos la API real de Twilio y la probamos. La **cuenta de prueba** de Twilio solo permite enviar **plantillas predefinidas** (el "body" es el nombre de la plantilla) y **no permite mensajes de voz propios**. Por eso la demo funciona así:

Canal	En la demo	Con una cuenta completa
SMS	**Real:** el celular suena con la plantilla de Twilio. El texto de PORTIX se muestra en el celular dibujado, declarado como simulado.	Sale el texto de PORTIX tal cual. Solo se vacía TWILIO_PLANTILLA_PRUEBA; el código no cambia.
Voz	**Simulada:** "Escuchar aquí" lo lee con la voz del navegador (Web Speech API, es-MX).	Llamada real con voz en español de México (Polly.Mia), ya programada.
Sin Twilio o si falla	El aviso queda "Simulado" y la pantalla dice por qué.	Igual.

8. Tecnologías que usamos y por qué

Tecnología	Para qué	Por qué la elegimos
React 18 + Vite	Frontend: tablero, vistas y formularios.	Componentes reutilizables y recarga instantánea. Vite compila rápido y reenvía /api al backend en desarrollo (sin CORS).
FastAPI (Python)	Backend: la API REST.	Simple de leer, valida datos automáticamente y genera documentación de la API. Python facilita la integración con IA.
Pydantic	Modelos de datos y validación.	Si llega un dato que no está en el contrato, responde 422 solo. Los mismos modelos le dicen a Gemini qué forma de JSON devolver.
JSON en memoria	Datos de la demo.	Sin base de datos: la demo siempre arranca igual y se reinicia con un botón. En producción sería PostgreSQL.
Gemini Flash (google-genai)	Leer documentos e interpretar mensajes.	Plan gratuito de Google AI Studio, lee PDFs e imágenes y responde JSON con esquema en pocos segundos.
Twilio (API REST)	SMS y llamadas al operador (giro).	El estándar para SMS y voz programables. Lo llamamos con la librería estándar de Python (urllib): sin dependencias extra.
Web Speech API	"Escuchar aquí": voz en el navegador.	Gratis, sin servidor, con voces en español.
pytest + httpx	46 pruebas automáticas.	Verifican cada regla del semáforo, los bordes de la ventana, la API y el giro, sin llamar a Gemini ni a Twilio.
Docker + Render	Despliegue en la nube.	Un solo servicio sirve la API y el frontend compilado: un solo link. Plan gratuito.
Claude Code	Desarrollo asistido.	Investigación, código por fases, pruebas y revisión; registrado en la bitácora de IA.

9. Arquitectura y archivos del código

```
Navegador (React + Vite)
   |  API REST (/api/...)
FastAPI (backend/main.py, servido por servidor.py junto con el frontend)
   |-- semaforo.py  reglas de color y estimación de tiempo
   |-- ia.py        Gemini: documentos, mensajes y datos del viaje
   |-- avisos.py    Twilio: SMS y voz al operador (giro)
   |-- modelos.py   formas de los datos (Pydantic)
   '-- data/*.json contenedores y congestión (sintéticos)
```

Backend (carpeta backend/)

Archivo	Qué contiene
main.py	Todos los endpoints. Guarda contenedores, chats, avisos y el contador en memoria. Reloj de la demo.
semaforo.py	Las reglas de color. Funciones puras: reciben datos y regresan un resultado (fáciles de probar).
ia.py	Todas las llamadas a Gemini, con esquemas, modelos alternos y respuestas de respaldo.
avisos.py	Textos del SMS y de la voz, y el envío con Twilio (real, plantilla de prueba o simulado).
modelos.py	Modelos Pydantic: contenedor, documento, resumen, aviso, etc.
servidor.py	Para el despliegue: sirve la API y el frontend compilado en un solo servicio.
data/	contenedores.json (9 contenedores sintéticos) y congestion.json (tabla simulada por hora).
tests/	46 pruebas con pytest: semáforo, API y avisos.

Frontend (carpeta frontend/src/)

Archivo	Qué contiene
App.jsx	Estado general, pestañas sincronizadas con la URL y recarga de datos.
api.js	Único punto de contacto con el backend: una función por endpoint.
components/vistas/	VistaPanel, VistaDocumentos, VistaChat, ListaCompacta, Checklist, AntesDespues.
components/	Encabezado, Resumen, GraficaCongestion, ListaContenedores, PanelDetalle, BarraVentana, SimularSalida, Reprogramar, SubirDocumento, ChatAgencia, AgregarContenedor, AvisoOperador.
textos.js	Etiquetas en español de los valores del contrato.

Endpoints de la API

Método y ruta	Para qué
GET /api/salud	Estado del servidor, hora de la demo, IA y modo de SMS (nunca regresa las keys).
POST /api/reiniciar	Regresa la demo a su estado inicial.
GET / POST /api/contenedores	Listar con color calculado / agregar un contenedor (201, 409 si existe, 422 si es inválido).
GET /api/contenedores/{id}	Detalle de un contenedor.
PATCH .../{id}/documentos	Cambiar el estado de un documento; regresa el color recalculado.
POST .../{id}/simular-salida	Probar otra hora de salida sin guardar.
POST .../{id}/reprogramar	Mover o cancelar la cita; suma No Show evitado y avisa al operador.
POST .../{id}/revisar-documento	Subir PDF o foto para que la IA lo revise.
GET / POST .../{id}/chat	Historial del chat / mensaje de la agencia que la IA interpreta.
GET .../{id}/avisos · POST .../{id}/avisar-operador	Historial y vista previa de avisos / mandar SMS o voz.
POST /api/extraer-viaje	La IA lee una carta porte para llenar el formulario.
GET /api/resumen · GET /api/congestion	Conteo por color, No Show, horas evitadas / tabla de congestión simulada.

El contrato completo (formas de cada petición y respuesta) está en docs/contrato-api.md.

10. Qué es real y qué es simulado

Regla del hackathon: presentar como funcional algo que no lo es descalifica. Todo lo simulado se declara en la pantalla y en el README.

Elemento	Estado
Tiempos del puerto y su desglose	**Real:** ITPAP de T21, septiembre 2026.
Reglas de cita (tolerancia y cargo de No Show)	**Reales:** avisos de ICAVE; pueden variar entre terminales.
Reglas del semáforo y cálculos	**Reales y funcionando** (con pruebas automáticas).
Revisión de documentos e interpretación de mensajes	**Reales con Gemini**; si falla, respuesta de respaldo etiquetada.
Contenedores, operadores, placas y teléfonos	**Sintéticos.**
Congestión por hora del día	**Simulada:** supuesto del equipo; el ITPAP solo publica un promedio mensual.
Horas de fila evitadas	**Estimación:** detenidos × espera promedio ITPAP (3 h 49 min).

Elemento	Estado
Hora actual de la demo	**Fija** (06:15) para que la demo sea reproducible.
Chat con la agencia	**Simulado** dentro de la app (no hay WhatsApp).
SMS al operador	**Envío real con Twilio**; con la cuenta de prueba sale la plantilla de Twilio y el texto de PORTIX se muestra simulado.
Llamada de voz	**Simulada** (voz del navegador); el código de la llamada real existe.
VUCEM, SAT, sistemas de citas de las terminales	**No existen** en el proyecto; se simulan con el checklist y el reprogramar.

11. Alcance: qué hace y qué no

Dentro del alcance (construido y funcionando)

- Semáforo con reglas explicables, congestión simulada por hora y llegada contra la ventana de la cita.
- Reprogramar o cancelar citas con contador de No Show evitados y pesos ahorrados.
- Checklist de 5 documentos, revisión de documentos con IA y chat con la agencia interpretado por IA.
- Agregar contenedores (a mano o llenando desde la carta porte con IA).
- KPIs de impacto: No Show evitados, pesos ahorrados y horas de fila evitadas (estimación).
- Avisos al operador por SMS (real con Twilio) y voz (navegador).
- Despliegue público, reinicio de la demo y 46 pruebas automáticas.

Fuera del alcance de esta versión

- Conexión real con VUCEM, el SAT o los sistemas de citas de las terminales.
- WhatsApp real (el chat es simulado).
- Un modelo predictivo entrenado: no hay datos públicos por hora; usamos reglas y congestión simulada.
- Rastreo GPS en tiempo real.
- Usuarios, login y varias empresas; base de datos (todo vive en memoria).
- Validación de documentos contra el SAT: solo revisamos consistencia entre el documento y el viaje.
- Coordinación entre varias transportistas o agencias (visión a futuro).

12. Empresas y organizaciones que podrían ayudar

Ninguna de estas organizaciones participa hoy en el proyecto. Son **aliados posibles** para llevar PORTIX de demo a operación real, según lo que cada una aporta.

Organización	Qué podría aportar	Para qué parte de PORTIX
Terminales del puerto (ICAVE, CICE y demás)	Acceso a su sistema de citas: horarios, folios, estado de la cita y cancelación.	Reprogramar y cancelar citas de verdad; ventana real por terminal.

Organización	Qué podría aportar	Para qué parte de PORTIX
ASIPONA Veracruz (administración del puerto)	Datos de flujo en accesos y garitas, coordinación entre actores del puerto.	Congestión real por hora en lugar de la simulada.
ANAM / VUCEM	Consulta del estatus del pedimento y de la carta porte.	Checklist con validación oficial, no solo de consistencia.
CANACAR (autotransporte)	Difusión entre transportistas y un piloto con empresas medianas.	Adopción y medición del impacto real.
Agencias aduanales (y su confederación, CAAAREM)	Canal oficial para recibir los avisos y responder.	Chat con la agencia por WhatsApp o integración directa.
T21 (ITPAP)	Datos más finos del tiempo en el puerto.	Mejores estimaciones de espera y de horas evitadas.
Proveedores de tecnología: Google (Gemini), Twilio u otros, Sintergica	IA y mensajería en producción (cuenta completa de SMS y voz).	Revisión de documentos, chat y avisos al operador a escala.

Modelo de adopción posible

- **Piloto:** una transportista mediana durante un mes, comparando No Show y horas de fila contra el mes anterior.
- **Quién paga:** la transportista, porque es quien ahorra (diésel, horas de operador y $999 por cada No Show).
- **Primera integración:** el sistema de citas de una terminal, porque es lo que convierte la sugerencia de reprogramar en una acción real.

13. Qué se necesita para correrlo, desplegarlo y llevarlo a producción

Para correrlo en una computadora

- Python 3.11 o más reciente y Node 20 o más reciente.
- Backend: `cd backend · python -m venv .venv · pip install -r requirements.txt · uvicorn main:app --reload`
- Frontend: `cd frontend · npm install · npm run dev` y abrir http://localhost:5173
- Pruebas: `pip install -r requirements-dev.txt · pytest`
- Opcional: backend/.env con GEMINI_API_KEY (sin ella, la IA usa respuestas de respaldo) y las variables de Twilio (sin ellas, los avisos se simulan). El .env nunca se sube al repo.

Para el despliegue (Render)

- El Dockerfile compila el frontend y corre el backend en un solo servicio.
- Variables de entorno en Render: GEMINI_API_KEY, GEMINI_MODEL, PORTIX_HORA_ACTUAL, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM, TELEFONO_DEMO y

TWILIO_PLANTILLA_PRUEBA.

- El plan gratuito duerme el servicio sin uso: abrir el link un par de minutos antes de presentar y presionar "Reiniciar demo".

Para llevarlo a producción

- Integración con los sistemas de citas de las terminales y con VUCEM.
- Base de datos (PostgreSQL), login y separación por empresa.
- Cuenta completa de Twilio (o proveedor local) para SMS con texto propio y llamadas reales.
- Datos reales de congestión por hora para reemplazar la tabla simulada.
- Aviso de privacidad y manejo de datos personales de operadores conforme a la ley mexicana de protección de datos.

14. Guion de la demo (unos 4 minutos)

Paso	Qué mostrar	Qué decir
1	Encabezado y resumen.	Son las 06:15 (hora fija). Hay 3 que salen, 4 que esperan y 2 que hay que reprogramar. Ya evitamos 15.3 horas de fila.
2	Clic en un contenedor rojo (TGHU o BMOU).	Su ventana cierra a las 06:45; aunque saliera ahora llega a las 07:00. Si no hacemos nada, paga $999.
3	Cancelar la cita.	Se suma un No Show evitado ($999) y al operador le llega un SMS: suena el celular real.
4	Aviso al operador: Escuchar aquí.	El operador no tiene smartphone: le llega un SMS de 160 caracteres sin acentos, o una llamada que deletrea el contenedor.
5	Vista Documentos: subir una carta porte con un error.	Gemini compara placas, contenedor y operador con el viaje y marca el error. El contenedor pasa a amarillo.
6	Vista Chat: "ya quedó pagado, al rato te paso el comprobante".	La IA entiende, pero el pedimento sigue pendiente hasta recibir el comprobante (regla estricta).
7	+ Agregar contenedor, llenar desde carta porte.	El despachador agrega el contenedor y PORTIX lo pone en el semáforo.
8	Cierre.	Todo lo simulado está etiquetado en pantalla. 46 pruebas automáticas respaldan las reglas.

15. Preguntas probables del jurado

¿Por qué este contenedor está en rojo?

Porque a la hora actual (06:15) ya pasó la última hora a la que podía salir y llegar antes del cierre de su ventana. Rojo solo cuando ya no alcanza; si todavía alcanza saliendo antes, es amarillo.

¿Qué pasa si Gemini se cae o se acaba el límite gratuito?

ia.py no truena: intenta otro modelo y, si todo falla, regresa una respuesta de respaldo con la misma forma. La pantalla muestra "Respuesta de respaldo" para que nadie la confunda con la IA real.

¿La congestión por hora es real?

No. El ITPAP solo publica promedios mensuales; la tabla por hora es un supuesto del equipo y está etiquetada como simulada en la gráfica y en el README.

¿De dónde salen las horas de fila evitadas?

Es una estimación: contenedores detenidos por documentos pendientes × 3 h 49 min, la espera promedio en Veracruz en septiembre de 2026 según el ITPAP. Lo dice el tooltip de la tarjeta.

¿El SMS es real?

El envío sí: sale por la API de Twilio y suena el celular. La cuenta de prueba solo permite plantillas de Twilio, así que el texto de PORTIX se muestra en pantalla y está declarado como simulado. Con una cuenta completa sale tal cual sin cambiar el código.

¿Por qué el SMS va sin acentos?

Una letra fuera de GSM-7, como la "á", cambia la codificación y el límite baja de 160 a 70 caracteres; el mensaje se partiría en varios SMS.

¿Por qué no hay base de datos ni login?

Están fuera del alcance a propósito: la demo debe arrancar siempre igual y reiniciarse con un botón. En producción serían PostgreSQL y login por empresa.

¿Cómo saben que funciona?

Hay 46 pruebas automáticas (pytest) que cubren cada regla del semáforo, los bordes exactos de la ventana, la API y el giro, sin depender de Gemini ni de Twilio.

¿Por qué el usuario es la transportista y no la agencia?

Porque la transportista hace la fila y paga el No Show. La agencia participa a través de los avisos.

¿Cómo cambiarían de proveedor de IA?

Solo se toca backend/ia.py; el resto del código llama a las mismas funciones y recibe la misma forma de datos.

16. Fuentes

- T21: Demoras portuarias cuestan más de 1,166 mdp mensuales al autotransporte en México (sep. 2026).
- T21: Puertos reducen 42% el costo de sus demoras para el autotransporte (oct. 2026).
- Plumas Libres: CANACAR denuncia demoras de hasta 1 día en la aduana de Veracruz (jun. 2026).
- InfoVeracruz: Trámites y congestionamientos frenan la competitividad logística del puerto de Veracruz (oct. 2026).
- ICAVE: Lineamientos ICA-2025-009, cargo de No Show.
- ICAVE: ICA-2022-013, tolerancia de citas.

Los enlaces completos están en la sección "Fuentes" del README del repositorio.